LEARN BEING FORWARD

Programming — C# Programming

OOP in C#

COMPLETE LEARNING GUIDE

What You'll Learn

C# was designed as an object-oriented language from the start, and its OOP features are both more structured and more opinionated than C++'s — properties replace manual getter/setter method pairs, every class implicitly inherits from object, and polymorphism requires an explicit, deliberate pairing of virtual and override rather than C++'s easier-to-misuse implicit virtual dispatch. This sub-module covers class anatomy and C#'s signature auto-implemented properties, constructors and object initializer syntax, inheritance with the sealed/virtual/override keyword triad, interfaces versus abstract classes and when to choose each, the mechanics of overriding versus overloading, access modifiers, and static members — building the same conceptual foundation as the C++ classes sub-module, but through C#'s more guided, compiler-enforced idioms.

1. Classes, Properties, and Auto-Implemented Properties

A property looks like a public field from the outside (object.Name) but is backed by get and set accessor methods, letting a class validate, compute, or restrict access to its internal state without changing how calling code reads or writes it — this is the idiomatic C# replacement for C++'s manual getName()/setName() method pairs. An auto-implemented property (public string Name { get; set; }) lets the compiler generate a hidden backing field automatically when no custom logic is needed, and set can be restricted to private (settable only inside the class) or omitted entirely combined with constructor assignment to create an effectively read-only property.

CSHARP

```
public class Person {
    public string Name { get; set; }             // full read/write auto-property
    public int Age { get; private set; }         // readable externally,
settable only inside the class

    public Person(string name, int age) {
        Name = name;
        Age = age;
    }

    public void HaveBirthday() => Age++; // can modify Age internally
}

var p = new Person("Asha", 30);
// p.Age = 40; // COMPILE ERROR — setter is private
p.HaveBirthday();
```

```
Console.WriteLine(p.Age); // 31
```

2. Constructors and Object Initializers

Constructors in C# work much like C++'s, running once when an object is created via new, and can be overloaded (multiple constructors with different parameter lists) with one constructor delegating to another via the this(...) initializer syntax to avoid duplicating setup logic. Object initializer syntax (new Person { Name = "Asha", Age = 30 }) sets public properties immediately after construction in a single expression, which is convenient for simple data-holding classes but still runs the parameterless constructor first — it is not a replacement for a constructor when validation logic or required fields are involved.

```csharp
public class Point {
    public int X { get; set; }
    public int Y { get; set; }

    public Point() : this(0, 0) {} // delegates to the other constructor
    public Point(int x, int y) { X = x; Y = y; }
}

var origin = new Point();                // uses the delegating constructor -> (0,0)
var p = new Point { X = 3, Y = 4 };      // object initializer syntax
var p2 = new Point(5, 6);                // direct constructor call
```

3. Inheritance: sealed, virtual, and override

A class inherits from exactly one base class in C# (no multiple class inheritance, though interfaces fill much of that gap — see below) using a colon (class Dog : Animal). Unlike C++, where every member function is implicitly overridable unless marked final, C# methods are NOT virtual by default — a base class method must be explicitly marked virtual to allow derived classes to override it with the override keyword, and attempting override on a non-virtual base method is a compile error rather than a silent new, unrelated method (which is closer to what C++ does when override is forgotten). sealed on a class prevents any further inheritance from it, and sealed on an override prevents further overriding of that specific method in classes derived even further down the hierarchy.

- A base method must be explicitly virtual (or abstract) before a derived class can override it — this is the opposite default from C++, and it's enforced by the compiler, not just convention.
- new (instead of override) on a derived method intentionally HIDES the base method rather than overriding it — a frequent, confusing beginner mistake when override was actually intended.

```csharp
public class Animal {
    public virtual void Speak() => Console.WriteLine("Some generic sound");
}

public class Dog : Animal {
    public override void Speak() => Console.WriteLine("Woof!");
}

Animal a = new Dog();
a.Speak(); // "Woof!" — dynamic dispatch via override, resolved by the OBJECT's real type
```

4. Interfaces vs. Abstract Classes

An interface declares a contract — a set of members any implementing class must provide — with no state (fields) and, traditionally, no implementation (default interface methods, added in C# 8, are a more recent, more limited exception); a class can implement any number of interfaces, which is how C# approximates multiple inheritance without its diamond-problem complications. An abstract class, by contrast, can hold real state (fields), provide some concrete (already-implemented) members alongside abstract ones that derived classes must implement, and a class can inherit from only ONE abstract (or any) class. The practical rule: use an interface to describe 'can do X' capabilities shared across otherwise-unrelated classes; use an abstract class when there's real shared state or behavior to inherit, not just a shared contract.

```csharp
public interface IFlyable {
    void Fly(); // no implementation, no state
}

public abstract class Bird {
    public string Name { get; set; } // real, shared state
    public abstract void MakeSound(); // must be implemented by derived classes
    public void Sleep() => Console.WriteLine($"{Name} is sleeping"); // shared, concrete behavior
}

public class Eagle : Bird, IFlyable {
    public override void MakeSound() => Console.WriteLine("Screech!");
    public void Fly() => Console.WriteLine($"{Name} soars high");
}
```

5. Polymorphism: Overriding vs. Overloading

Overriding (override) provides a NEW implementation of an already-virtual base class member with the EXACT SAME signature, resolved at runtime based on the object's actual type — this is true polymorphism. Overloading defines MULTIPLE methods with the SAME NAME but DIFFERENT parameter lists (different types or counts) within the same class, resolved at COMPILE time based on the arguments supplied at the call site — this is a convenience feature for naming related operations consistently, not a form of runtime polymorphism at all, and confusing the two (expecting overload resolution to behave

dynamically, or expecting override to allow a different signature) is a common beginner mix-up.

```csharp
public class Calculator {
    public int Add(int a, int b) => a + b;           // overload 1
    public double Add(double a, double b) => a + b;  // overload 2 — resolved at COMPILE time
}

var calc = new Calculator();
Console.WriteLine(calc.Add(2, 3));     // 5    — calls the int overload
Console.WriteLine(calc.Add(2.5, 3.5)); // 6.0  — calls the double overload
```

6. Access Modifiers

public members are accessible from anywhere; private members (the default for class members if unspecified) are accessible only within the same class; protected members are accessible within the class and any class deriving from it; internal members are accessible anywhere within the same assembly (compiled project) but not from other assemblies/projects; protected internal combines protected and internal (accessible to derived classes OR anything in the same assembly); and private protected (accessible only to derived classes within the same assembly) is the narrowest combination. Choosing the narrowest access level that still does the job is standard practice, since it minimizes the surface area other code can depend on and later break when you refactor.

7. Static Members and Static Classes

A static member (field, property, or method) belongs to the TYPE itself rather than to any individual instance — there's exactly one copy of a static field shared across every instance of the class, and static methods are called on the type name directly (Math.Sqrt(4)) rather than on an object instance. A static class (the whole class marked static) cannot be instantiated at all and can contain only static members — used for pure utility/helper classes with no per-instance state, like the built-in Math and Console classes. Overusing static state for things that should actually be per-instance data is a common design mistake, since it creates hidden, shared mutable state that's easy to accidentally corrupt from unrelated parts of a program.

```csharp
public static class MathUtils {
    public static int Square(int x) => x * x; // no instance needed — called on the type itself
}

public class Counter {
    private static int totalInstancesCreated = 0; // shared across ALL Counter instances
    public Counter() { totalInstancesCreated++; }
    public static int TotalInstances => totalInstancesCreated;
}

Console.WriteLine(MathUtils.Square(5)); // 25
new Counter(); new Counter();
Console.WriteLine(Counter.TotalInstances); // 2
```

Common Pitfalls

- Forgetting that C# methods are non-virtual by default — a base method must be marked virtual before override on a derived method is even legal.
- Using new instead of override on a derived method, which hides the base method instead of truly overriding it (and breaks polymorphic dispatch through a base reference).
- Choosing an interface when a class actually needs to share real, concrete state across its implementers — that calls for an abstract class instead.
- Expecting overload resolution to behave polymorphically at runtime, when it is actually resolved entirely at compile time based on static argument types.
- Making a field or property public when a narrower access level (private, protected, internal) would fully satisfy every real caller's needs.
- Storing per-instance data in a static field, creating shared mutable state that's silently corrupted by unrelated instances.

Key Takeaways

- Properties (get/set) are C#'s idiomatic replacement for manual getter/setter method pairs, and auto-properties remove boilerplate when no custom logic is needed.
- C# members are non-virtual by default; virtual/override must both be explicit, and the compiler enforces the pairing (unlike C++'s easier-to-misuse implicit virtuality).
- Use interfaces for 'can do X' contracts shared across unrelated classes; use abstract classes when real shared state or concrete behavior needs to be inherited.
- Overriding is resolved at runtime based on the object's real type (true polymorphism); overloading is resolved at compile time based on argument types (not polymorphism).
- Always choose the narrowest access modifier that satisfies real callers, and avoid static fields for state that is genuinely per-instance.

Learn More

Microsoft Learn — Object-oriented programming (C#)
https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/

Microsoft Learn — Interfaces
https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/interfaces